
EE542 Lab 6: High-Performance CUDA Programming on a Cloud GPU

Huacong Tang
University of Southern California

Xiao Yang
University of Southern California

Jiachen Lu
University of Southern California

 [Code](#)  [Demo Video](#)

1 Matrix multiplication on the CPU

The baseline is the handout’s single-precision matrix multiply in C, run on a local Apple M4 Pro and again on the cloud instance (Table 1, Figure 1).

N	M4 Pro, local (s)	cloud host (s)
256	0.0097	0.0129
512	0.0886	0.246
768	0.316	0.375
1024	0.898	3.123
1536	2.863	8.915
2048	10.5	40.4
3072	34.2	169.4

Table 1: CPU matrix multiply, `gcc -O2`, one thread. Median of three runs.

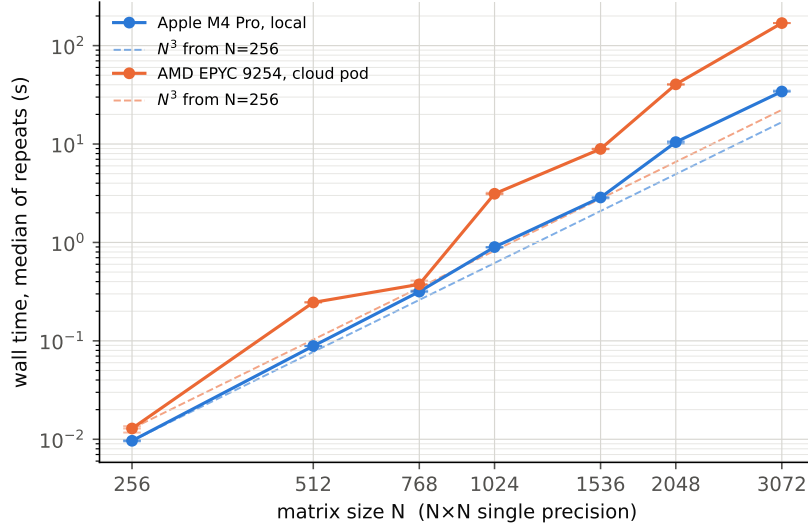


Figure 1: CPU wall time versus N on both machines. Dashed lines are N^3 scaling.

2 Naive CUDA kernel

We implemented the handout’s naive kernel with 16×16 thread blocks. Apple Silicon has no CUDA support, so all GPU runs were on the cloud instance below.

3 Running CUDA on a cloud instance

The instance is a RunPod pod with one NVIDIA L4 running Ubuntu 22.04 and CUDA 12.8. The naive kernel was run at the same seven sizes as the CPU (Table 2, Figure 2), and its results match the CPU’s at every size.

N	Kernel (ms)	H2D copy (ms)	D2H copy (ms)	Total (ms)
256	0.028	0.064	0.162	0.255
512	0.158	0.210	0.623	0.991
768	0.506	0.417	1.259	2.181
1024	1.170	0.704	2.160	4.034
1536	3.905	1.510	4.792	10.21
2048	9.287	2.639	8.544	20.47
3072	37.12	5.852	19.24	62.21

Table 2: Naive CUDA kernel on the L4. H2D and D2H are the copies to and from the GPU. Median of three runs.

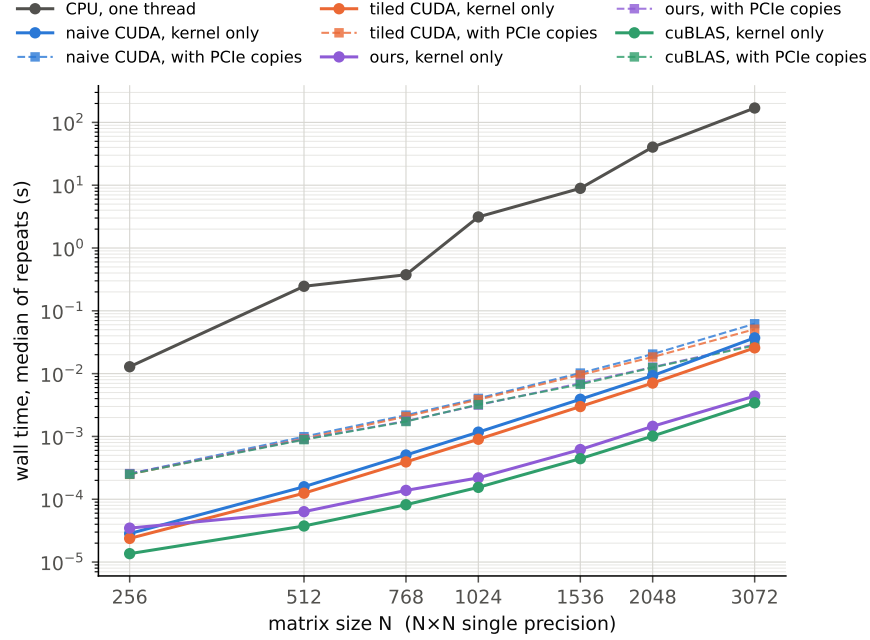


Figure 2: Wall time versus N on the cloud instance for the CPU and the three GPU versions, with and without the copies.

4 Optimizing CUDA code

The tiled kernel is the handout’s shared-memory version with `TILE_WIDTH 16`, run at the same sizes on the same instance (Table 3).

N	Kernel (ms)	H2D copy (ms)	D2H copy (ms)	Total (ms)
256	0.024	0.065	0.160	0.249
512	0.125	0.210	0.589	0.924
768	0.393	0.423	1.253	2.069
1024	0.902	0.699	2.258	3.859
1536	2.991	1.520	5.028	9.539
2048	7.079	2.641	8.663	18.38
3072	25.73	5.854	19.32	50.90

Table 3: Tiled CUDA kernel on the L4. Median of three runs.

5 Performance comparison

Table 4 gives the execution times in the handout’s layout and Table 5 the speedup over the CPU. Once the copies to and from the GPU are counted, the speedup drops by about an order of magnitude.

N	256	512	768	1024	1536	2048	3072
CPU (C), s	0.0129	0.246	0.375	3.123	8.915	40.4	169.4
Naive CUDA, ms	0.028	0.158	0.506	1.170	3.905	9.287	37.12
Tiled CUDA, ms	0.024	0.125	0.393	0.902	2.991	7.079	25.73
cuBLAS, ms	0.014	0.037	0.082	0.155	0.443	1.014	3.430
Ours (Part 6), ms	0.035	0.063	0.138	0.219	0.618	1.448	4.392

Table 4: Execution time by implementation and matrix size. GPU rows are the kernel only.

N	256	512	768	1024	1536	2048	3072
Naive, kernel only	453	1553	742	2669	2283	4350	4564
Naive, with copies	51	248	172	774	873	1974	2723
Naive, copies' share	89%	84%	77%	71%	62%	55%	40%
Tiled, kernel only	541	1968	955	3463	2981	5708	6583
Tiled, with copies	52	266	181	809	935	2198	3328
Tiled, copies' share	90%	86%	81%	77%	69%	61%	49%
cuBLAS, kernel only	946	6553	4588	20189	20143	39851	49382
cuBLAS, with copies	51	276	217	969	1319	3231	6041
cuBLAS, copies' share	95%	96%	95%	95%	93%	92%	88%
Ours, kernel only	371	3876	2714	14262	14426	27898	38569
Ours, with copies	50	272	213	990	1265	3181	5835
Ours, copies' share	86%	93%	92%	93%	91%	89%	85%

Table 5: Speedup over the CPU, for the kernel alone and with the copies.

6 Continued improvements

6.1 cuBLAS

The cuBLAS version replaces our kernel with the handout's `cublasSgemm` call (Table 6).

N	Kernel (ms)	H2D copy (ms)	D2H copy (ms)	Total (ms)
256	0.014	0.081	0.155	0.250
512	0.037	0.254	0.598	0.889
768	0.082	0.457	1.190	1.728
1024	0.155	0.729	2.340	3.223
1536	0.443	1.535	4.780	6.757
2048	1.014	2.646	8.847	12.51
3072	3.430	5.844	18.77	28.04

Table 6: `cublasSgemm` on the L4. Median of three runs.

6.2 Analysis questions

1. How does performance change as matrix size increases? The CPU slows down faster than N^3 because the matrices stop fitting in its cache. The GPU gets more efficient as N grows, so the speedup of the naive kernel rises from 450 at $N = 256$ to 4600 at $N = 3072$.

2. At what point does the GPU significantly outperform the CPU? At every size we measured. Even at $N = 256$ the naive kernel is 450 times faster than the CPU, and 50 times faster with the copies included.

3. How much speedup is gained by tiling optimization vs. naive CUDA? The tiled kernel is 1.2 to 1.45 times faster than the naive one. The gain is small because the L4's cache already serves most of the naive kernel's reads.

4. How close is your optimized kernel to cuBLAS performance? Our best kernel (Section 6.3) reaches 70–78% of cuBLAS speed from $N = 1024$ up. At $N = 256$ cuBLAS is 2.5 times faster.

5. Why might cuBLAS still outperform hand-written kernels? cuBLAS is tuned for each GPU architecture and picks a different kernel for each problem size. Our kernel has one fixed design and lacks its finer-grained optimizations.

6.3 Further improvement

Following Boehm [1], we wrote a kernel in which each thread computes an 8×8 patch of the output, using larger tiles and vectorized loads. It is 4 to 6 times faster than the tiled kernel from $N = 1024$ up (Table 7).

N	Kernel (ms)	H2D copy (ms)	D2H copy (ms)	Total (ms)
256	0.035	0.063	0.159	0.256
512	0.063	0.209	0.630	0.902
768	0.138	0.413	1.210	1.761
1024	0.219	0.696	2.240	3.155
1536	0.618	1.514	4.918	7.050
2048	1.448	2.635	8.619	12.70
3072	4.392	5.844	18.79	29.03

Table 7: Our optimized kernel on the L4. Median of three runs.

7 Shared library and Python

We built the handout’s `matrix_lib.cu` into `libmatrix.so` and added a second function that runs the optimized kernel of Section 6.3. The Python script calls both through `ctypes` and compares them with numpy’s `A @ B` (Table 8). The results agree with numpy, and the first call takes an extra 276 ms to start CUDA.

N	tiled call	fast call	alloc+free	H2D	kernel	D2H	numpy
256	3.95	0.39	0.23	0.06	0.04	0.04	1.23
512	0.98	0.85	0.42	0.21	0.07	0.12	15.3
768	1.81	1.56	0.70	0.41	0.14	0.22	289.7
1024	2.93	2.15	0.70	0.69	0.22	0.37	260.9
1536	6.68	3.95	1.02	1.52	0.62	0.77	257.8
2048	12.7	7.02	1.45	2.66	1.43	1.33	201.9
3072	36.3	28.7	2.39	5.85	4.38	2.96	486.2

Table 8: Python calls into `libmatrix.so`, milliseconds. The four middle columns are the stages of the fast call; “numpy” is `A @ B` on the CPU.

8 Convolution in the shared library

8.1 The C function and the test material

The C function convolves an $M \times M$ integer image with an $N \times N$ integer filter. We tested it with Sobel, Laplacian, Gaussian and Laplacian-of-Gaussian filters on three sample images from `scikit-image` (Figure 3). Every output is identical to `scipy.ndimage.convolve`.

8.2 The CUDA port

We ported the function to two CUDA kernels, a naive one and a tiled one that uses shared memory (Table 9). Both give the same output as the C program. The naive kernel is faster for small filters and the tiled kernel for large ones.

8.3 The library and the three-way comparison

We added the tiled kernel to `libmatrix.so` in two forms: a one-shot call, and a call that keeps the image on the GPU so several filters can be applied without uploading it again. Table 10 compares the C program, the CUDA executable and the Python program through the library. All outputs are identical, and keeping the image on the GPU is the fastest.

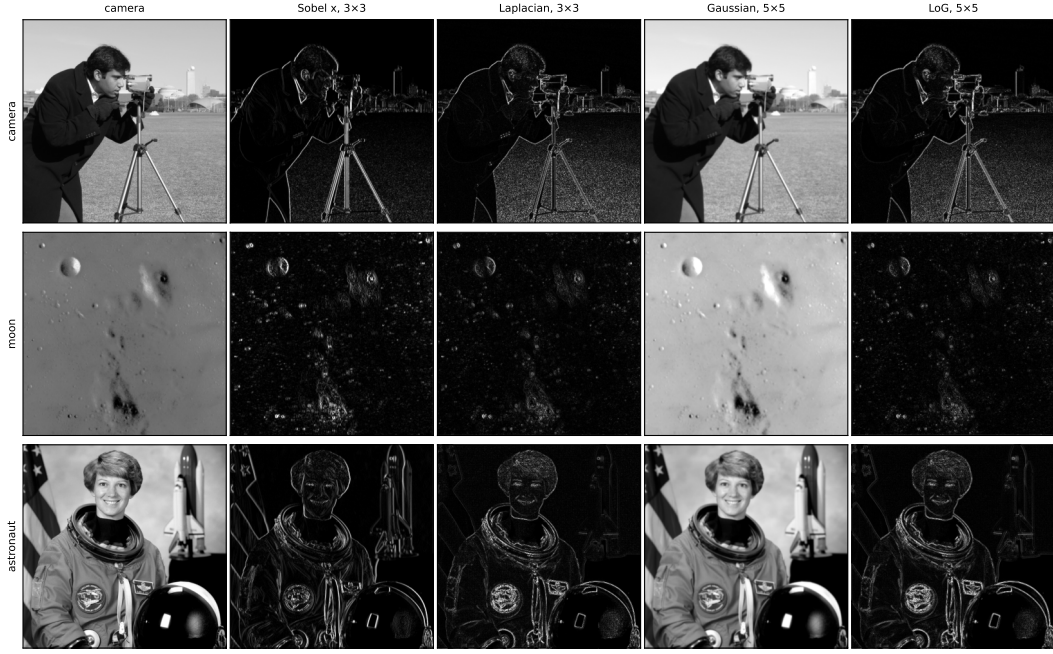


Figure 3: The three sample images (left) and their convolution with Sobel x , Laplacian, 5×5 Gaussian and 5×5 Laplacian of Gaussian.

M	N	CPU (s)	naive (ms)	tilted (ms)	tilted/naive	CPU/naive	CPU/tilted	equal
512	3	0.0026	0.012	0.014	0.83	220	183	yes
512	5	0.0057	0.018	0.018	0.96	320	306	yes
512	7	0.0100	0.028	0.024	1.14	364	415	yes
512	11	0.0230	0.057	0.041	1.40	404	565	yes
512	15	0.0424	0.095	0.065	1.46	448	656	yes
1024	3	0.0105	0.031	0.041	0.76	336	254	yes
1024	5	0.0227	0.058	0.058	1.01	390	394	yes
1024	7	0.0408	0.094	0.080	1.17	433	507	yes
1024	11	0.0955	0.200	0.136	1.47	478	702	yes
1024	15	0.170	0.350	0.216	1.62	487	786	yes
2048	3	0.0416	0.105	0.135	0.78	395	308	yes
2048	5	0.0906	0.206	0.208	0.99	439	436	yes
2048	7	0.162	0.346	0.290	1.19	468	559	yes
2048	11	0.376	0.768	0.500	1.54	489	752	yes
2048	15	0.687	1.369	0.833	1.64	502	825	yes
4096	3	0.173	0.528	0.840	0.63	327	206	yes
4096	5	0.364	0.927	1.094	0.85	392	332	yes
4096	7	0.650	1.547	1.377	1.12	421	473	yes
4096	11	1.495	3.377	2.239	1.51	443	668	yes
4096	15	2.736	5.945	3.599	1.65	460	760	yes

Table 9: Convolution by image size M and filter size N : the C program (seconds) and the two CUDA kernels (milliseconds, kernel only).

M	N	C exe (ms)	CUDA exe (ms)	Py one-shot (ms)	Py resident (ms)	scipy (ms)	equal
512	3	2.596	0.267	0.628	0.152	4.166	yes
512	7	10.22	0.277	2.506	0.196	12.25	yes
512	15	42.94	0.319	1.065	0.266	67.91	yes
1024	3	11.96	0.780	2.609	0.460	34.67	yes
1024	7	40.89	0.816	2.186	0.502	58.25	yes
1024	15	174.3	0.944	2.194	0.637	184.4	yes
2048	3	41.95	2.806	5.718	1.508	75.57	yes
2048	7	165.7	2.960	6.347	1.652	171.0	yes
2048	15	699.2	3.492	9.610	2.221	661.5	yes
4096	3	180.0	11.24	23.27	6.078	213.3	yes
4096	7	661.4	11.76	20.13	6.598	582.7	yes
4096	15	2811.6	13.89	22.38	8.777	2642.0	yes

Table 10: Three-way comparison on the camera image, milliseconds: the C program, the CUDA executable, Python through the library (one-shot and with the image resident), and `scipy.ndimage.convolve`.

References

- [1] S. Boehm, “How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog,” Dec. 2022. <https://siboehm.com/articles/22/CUDA-MMM>